

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверила:

Андренко К.В.

Цель работы: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС.

Ход работы

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Оптимизатор	Предобученная архитектура
9	CIFAR-100	Adam	DenseNet121

Код программы:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
import torchvision.transforms as transforms

import numpy as np
import matplotlib.pyplot as plt
import random
from PIL import Image
from collections import defaultdict

from torchvision.models import densenet121, DenseNet121_Weights

def main():
    SEED = 42
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("Используемое устройство:", device)
```

```

CLASSES = ('apple', 'aquarium_fish', 'baby', 'bear', 'beaver', 'bed', 'bee', 'beetle',
            'bicycle', 'bottle', 'bowl', 'boy', 'bridge', 'bus', 'butterfly', 'camel',
            'can', 'castle', 'caterpillar', 'cattle', 'chair', 'chimpanzee', 'clock',
            'cloud', 'cockroach', 'couch', 'crab', 'crocodile', 'cup', 'dinosaur',
            'dolphin', 'elephant', 'flatfish', 'forest', 'fox', 'girl', 'hamster',
            'house', 'kangaroo', 'keyboard', 'lamp', 'lawn_mower', 'leopard', 'lion',
            'lizard', 'lobster', 'man', 'maple_tree', 'motorcycle', 'mountain', 'mouse',
            'mushroom', 'oak_tree', 'orange', 'orchid', 'otter', 'palm_tree', 'pear',
            'pickup_truck', 'pine_tree', 'plain', 'plate', 'poppy', 'porcupine',
            'possum', 'rabbit', 'raccoon', 'ray', 'road', 'rocket', 'rose', 'sea',
            'seal', 'shark', 'shrew', 'skunk', 'skyscraper', 'snail', 'snake',
            'spider', 'squirrel', 'streetcar', 'sunflower', 'sweet_pepper', 'table',
            'tank', 'telephone', 'television', 'tiger', 'tractor', 'train', 'trout',
            'tulip', 'turtle', 'wardrobe', 'whale', 'willow_tree', 'wolf', 'woman',
            'worm')

NUM_CLASSES = 100

MEAN = (0.5071, 0.4865, 0.4409)
STD = (0.2673, 0.2564, 0.2762)

transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

train_set = torchvision.datasets.CIFAR100(root='./data', train=True,
                                           download=True, transform=transform_train)
test_set = torchvision.datasets.CIFAR100(root='./data', train=False,
                                           download=True, transform=transform_test)

BATCH_SIZE = 128
train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
test_loader = DataLoader(test_set, batch_size=256, shuffle=False, num_workers=2)

print(f"Train: {len(train_set)}, Test: {len(test_set)}, размер изображения: {train_set[0][0].shape}")

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, num_classes)
        self.dropout = nn.Dropout(0.4)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = SimpleCNN().to(device)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(model)

```

```

print(f"Обучаемых параметров: {n_params:,}")

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
N_EPOCHS = 8

history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    context = torch.enable_grad() if train else torch.no_grad()
    with context:
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            if train:
                optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            if train:
                loss.backward()
                optimizer.step()
            total_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += labels.size(0)
    return total_loss / total, correct / total

for epoch in range(1, N_EPOCHS + 1):
    train_loss, train_acc = run_epoch(train_loader, train=True)
    test_loss, test_acc = run_epoch(test_loader, train=False)
    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["test_loss"].append(test_loss)
    history["test_acc"].append(test_acc)
    print(f"Эпоха {epoch:2d}/{N_EPOCHS} | train loss {train_loss:.4f} acc {train_acc*100:5.2f}% | "
          f"test loss {test_loss:.4f} acc {test_acc*100:5.2f}%")

custom_model = model

IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)

transform_train_pt = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

transform_test_pt = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

train_set_pt = torchvision.datasets.CIFAR100(root='./data', train=True,
                                              download=True, transform=transform_train_pt)
test_set_pt = torchvision.datasets.CIFAR100(root='./data', train=False,
                                              download=True, transform=transform_test_pt)

BATCH_SIZE_PT = 64
train_loader_pt = DataLoader(train_set_pt, batch_size=BATCH_SIZE_PT, shuffle=True,
                             num_workers=2)
test_loader_pt = DataLoader(test_set_pt, batch_size=128, shuffle=False, num_workers=2)

print(f"Train: {len(train_set_pt)}, Test: {len(test_set_pt)}, размер изображения: {train_set_pt[0][0].shape}")

weights = DenseNet121_Weights.IMAGENET1K_V1

```

```

pretrained_model = densenet121(weights=weights)

for param in pretrained_model.features.parameters():
    param.requires_grad = False

in_features = pretrained_model.classifier.in_features
pretrained_model.classifier = nn.Linear(in_features, NUM_CLASSES)

pretrained_model = pretrained_model.to(device)

n_params_total = sum(p.numel() for p in pretrained_model.parameters())
n_params_trainable = sum(p.numel() for p in pretrained_model.parameters() if p.requires_grad)
print(pretrained_model.classifier)
print(f"Всего параметров: {n_params_total:,}")
print(f"Обучаемых параметров (только классификатор): {n_params_trainable:,}")

criterion_pt = nn.CrossEntropyLoss()
optimizer_pt = optim.Adam(filter(lambda p: p.requires_grad, pretrained_model.parameters()),
lr=1e-3)

N_EPOCHS_PT = 3
history_pt = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

def run_epoch_pt(loader, train=True):
    pretrained_model.train() if train else pretrained_model.eval()
    total_loss, correct, total = 0.0, 0, 0
    context = torch.enable_grad() if train else torch.no_grad()
    with context:
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            if train:
                optimizer_pt.zero_grad()
            outputs = pretrained_model(images)
            loss = criterion_pt(outputs, labels)
            if train:
                loss.backward()
                optimizer_pt.step()
            total_loss += loss.item() * images.size(0)
            correct += (outputs.argmax(1) == labels).sum().item()
            total += labels.size(0)
    return total_loss / total, correct / total

for epoch in range(1, N_EPOCHS_PT + 1):
    train_loss, train_acc = run_epoch_pt(train_loader_pt, train=True)
    test_loss, test_acc = run_epoch_pt(test_loader_pt, train=False)
    history_pt["train_loss"].append(train_loss)
    history_pt["train_acc"].append(train_acc)
    history_pt["test_loss"].append(test_loss)
    history_pt["test_acc"].append(test_acc)
    print(f"Эпоха {epoch:2d}/{N_EPOCHS_PT} | train loss {train_loss:.4f} acc {train_acc*100:5.2f}% | "
          f"test loss {test_loss:.4f} acc {test_acc*100:5.2f}%")

epochs_range_pt = range(1, N_EPOCHS_PT + 1)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(epochs_range_pt, history_pt["train_loss"], label="Train loss", marker='o',
markersize=3)
axes[0].plot(epochs_range_pt, history_pt["test_loss"], label="Test loss", marker='o',
markersize=3)
axes[0].set_xlabel("Эпоха"); axes[0].set_ylabel("Loss")
axes[0].set_title("DenseNet121: изменение ошибки"); axes[0].legend(); axes[0].grid(alpha=0.3)

axes[1].plot(epochs_range_pt, [a*100 for a in history_pt["train_acc"]], label="Train acc",
marker='o', markersize=3)
axes[1].plot(epochs_range_pt, [a*100 for a in history_pt["test_acc"]], label="Test acc",
marker='o', markersize=3)
axes[1].set_xlabel("Эпоха"); axes[1].set_ylabel("Accuracy, %")
axes[1].set_title("DenseNet121: изменение точности"); axes[1].legend();
axes[1].grid(alpha=0.3)

```

```

plt.tight_layout()
plt.show()

print(f"Итоговая точность DenseNet121 на тесте: {history_pt['test_acc'][-1]*100:.2f}%")

print("Сравнение результатов:")
print(f"Кастомная СНС: {history['test_acc'][-1]*100:.2f}% ({n_params:,} параметров, все обучаемые)")
print(f"DenseNet121: {history_pt['test_acc'][-1]*100:.2f}% ({n_params_trainable:,} обучаемых параметров из {n_params_total:,})")

print("\n" + "=" * 70)
print("СПРАВНЕНИЕ С SOTA НА CIFAR-100")
print("=" * 70)

your_results = {
    "SimpleCNN (ваша)": {"acc": history['test_acc'][-1] * 100, "params": n_params,
    "note": "с нуля, 8 эпох"},
    "DenseNet121 (ваша)": {"acc": history_pt['test_acc'][-1] * 100, "params":
n_params_trainable, "note": "только классификатор, 3 эпохи"},
}

fair_baselines = {
    "VGG-16": {"acc": 72.7, "params": 20_000_000, "note": "с нуля"},
    "ResNet-18": {"acc": 75.6, "params": 11_000_000, "note": "с нуля"},
    "ResNet-56": {"acc": 76.5, "params": 24_000_000, "note": "с нуля"},
    "Wide ResNet-28-10": {"acc": 81.2, "params": 36_000_000, "note": "с аугментацией"},
}

sota_pretrained = {
    "DenseNet-BC (k=24)": {"acc": 82.8, "params": 27_000_000, "note": "ImageNet pretrain"},
    "EfficientNet-B7": {"acc": 87.3, "params": 66_000_000, "note": "ImageNet pretrain"},
    "FixResNeXt-101": {"acc": 89.9, "params": 84_000_000, "note": "ImageNet pretrain"},
    "ViT-L/16": {"acc": 93.0, "params": 307_000_000, "note": "JFT-300M pretrain"},
    "ViT-H/14": {"acc": 93.8, "params": 632_000_000, "note": "JFT-300M pretrain"},
}

def print_table(title, data):
    print(f"\n--- {title} ---")
    print(f"{'Модель':<28}{'Точность':>10}{'Параметры':>15} Примечание")
    print("-" * 75)
    for name, info in data.items():
        print(f"{'name':<28}{'info['acc']':>9.2f}%{'info['params']':>15,} {'info['note']}'")

print_table("Ваши модели", your_results)
print_table("Честные бейзлайны (обучение с нуля)", fair_baselines)
print_table("SOTA с предобучением (нельзя сравнивать напрямую)", sota_pretrained)

all_models = {}
all_models.update(your_results)
all_models.update(fair_baselines)
all_models.update(sota_pretrained)

names = list(all_models.keys())
accs = [all_models[n]['acc'] for n in names]
colors = (
    ['crimson'] * len(your_results) +
    ['steelblue'] * len(fair_baselines) +
    ['seagreen'] * len(sota_pretrained)
)

fig, ax = plt.subplots(figsize=(12, 6))
bars = ax.barh(names, accs, color=colors)
ax.set_xlim(0, 102)
ax.set_xlabel("Точность на CIFAR-100, %")
ax.set_title("Сравнение ваших моделей с бейзлайнами и SOTA (CIFAR-100)")
ax.invert_yaxis()
ax.grid(axis='x', alpha=0.3)

```

```

for bar, acc in zip(bars, accs):
    ax.text(acc + 0.3, bar.get_y() + bar.get_height()/2,
            f"{acc:.2f}%", va='center', fontsize=9)

from matplotlib.patches import Patch
legend_elems = [
    Patch(facecolor='crimson', label='Ваши модели'),
    Patch(facecolor='steelblue', label='Бейзлайны (с нуля)'),
    Patch(facecolor='seagreen', label='SOTA (с предобучением)'),
]
ax.legend(handles=legend_elems, loc='center right')
plt.tight_layout()
plt.show()

your_best = max(your_results.values(), key=lambda x: x['acc'])['acc']
fair_best = max(fair_baselines.values(), key=lambda x: x['acc'])['acc']
sota_best = max(sota_pretrained.values(), key=lambda x: x['acc'])['acc']

print("\n" + "=" * 70)
print("ИТОГИ")
print("=" * 70)
print(f"Лучшая ваша модель: {your_best:.2f}%")
print(f"Лучший честный бейзлайн: {fair_best:.2f}% (отставание: {fair_best - your_best:+.2f} п.п.)")
print(f"Лучший SOTA (с предобучением): {sota_best:.2f}% (отставание: {sota_best - your_best:+.2f} п.п.)")
print("\nПримечание: SOTA-модели используют предобучение на")
print("ImageNet/JFT-300M, поэтому их сравнение с моделями,")
print("обученными с нуля, не является строгим.")

def predict_with_both(path):
    img = Image.open(path).convert('RGB')

    preprocess_custom = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ])
    preprocess_pt = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    ])

    custom_model.eval()
    pretrained_model.eval()

    with torch.no_grad():
        probs_custom = F.softmax(custom_model(preprocess_custom(img).unsqueeze(0).to(device)),
dim=1).cpu().numpy().flatten()
        probs_pt = F.softmax(pretrained_model(preprocess_pt(img).unsqueeze(0).to(device)),
dim=1).cpu().numpy().flatten()

        pred_custom = int(np.argmax(probs_custom))
        pred_pt = int(np.argmax(probs_pt))

    fig, axes = plt.subplots(1, 3, figsize=(15, 4), gridspec_kw={'width_ratios': [1, 1.2, 1.2]})
    axes[0].imshow(img); axes[0].axis('off'); axes[0].set_title("Входное изображение")

    axes[1].barh(CLASSES, probs_custom * 100, color=['red' if i == pred_custom else 'steelblue' for i in range(NUM_CLASSES)])
    axes[1].set_xlim(0, 100); axes[1].invert_yaxis()
    axes[1].set_title(f"Кастомная CHC: {CLASSES[pred_custom]}")
    axes[1].set_xlabel("Вероятность, %")

    axes[2].barh(CLASSES, probs_pt * 100, color=['green' if i == pred_pt else 'steelblue' for i in range(NUM_CLASSES)])
    axes[2].set_xlim(0, 100); axes[2].invert_yaxis()

```

```

axes[2].set_title(f"DenseNet121: {CLASSES[pred_pt]}")
axes[2].set_xlabel("Вероятность, %")

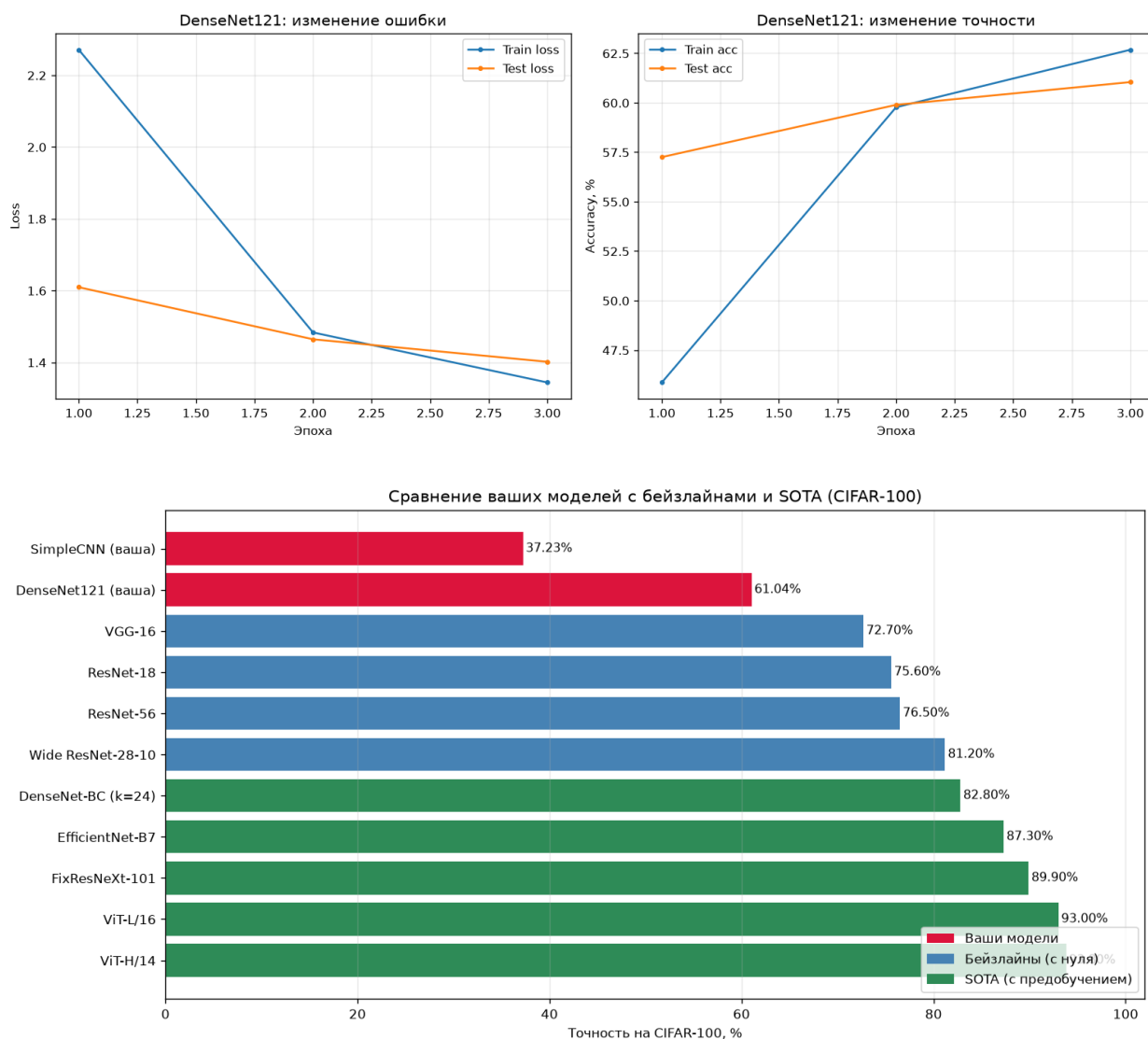
plt.tight_layout()
plt.show()
return pred_custom, pred_pt

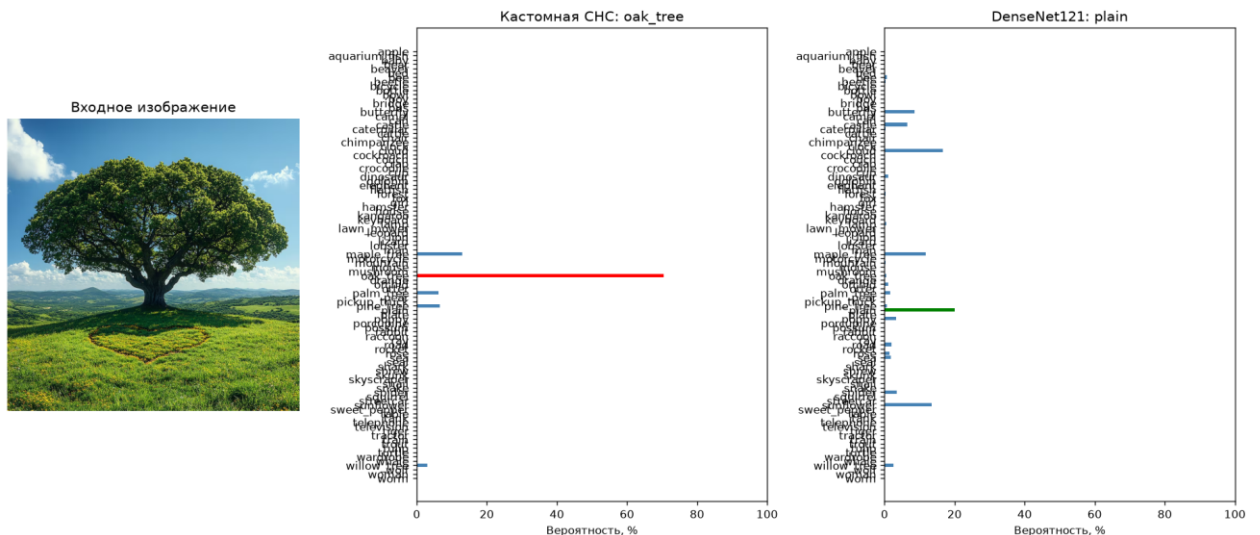
predict_with_both(r'C:\Users\admin\Desktop\test1.png')

if __name__ == '__main__':
    main()

```

Результат программы:





Используемое устройство: cpu

Train: 50000, Test: 10000, размер изображения: torch.Size([3, 32, 32])

SimpleCNN(

(conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

(fc1): Linear(in_features=2048, out_features=256, bias=True)

(fc2): Linear(in_features=256, out_features=100, bias=True)

(dropout): Dropout(p=0.4, inplace=False)

)

Обучаемых параметров: 643,492

Эпоха 1/8 | train loss 4.0635 acc 7.29% | test loss 3.5557 acc 15.78%

Эпоха 2/8 | train loss 3.5484 acc 15.01% | test loss 3.1749 acc 22.20%

Эпоха 3/8 | train loss 3.2797 acc 19.94% | test loss 2.9339 acc 26.85%

Эпоха 4/8 | train loss 3.1174 acc 22.92% | test loss 2.8027 acc 29.89%

Эпоха 5/8 | train loss 2.9858 acc 25.41% | test loss 2.6895 acc 31.18%

Эпоха 6/8 | train loss 2.8852 acc 27.26% | test loss 2.5122 acc 34.58%

Эпоха 7/8 | train loss 2.8048 acc 28.80% | test loss 2.5065 acc 35.29%

Эпоха 8/8 | train loss 2.7209 acc 30.53% | test loss 2.4014 acc 37.23%

Train: 50000, Test: 10000, размер изображения: torch.Size([3, 224, 224])

Linear(in_features=1024, out_features=100, bias=True)

Всего параметров: 7,056,356

Обучаемых параметров (только классификатор): 102,500

Эпоха 1/3 | train loss 2.2708 acc 45.86% | test loss 1.6093 acc 57.25%

Эпоха 2/3 | train loss 1.4832 acc 59.77% | test loss 1.4643 acc 59.89%

Эпоха 3/3 | train loss 1.3440 acc 62.67% | test loss 1.4015 acc 61.04%

Итоговая точность DenseNet121 на тесте: 61.04%

Сравнение результатов:

Кастомная СНС: 37.23% (643,492 параметров, все обучаемые)

DenseNet121: 61.04% (102,500 обучаемых параметров из 7,056,356)

СРАВНЕНИЕ С SOTA НА CIFAR-100

--- Ваши модели ---

Модель	Точность	Параметры	Примечание
SimpleCNN (ваша)	37.23%	643,492	с нуля, 8 эпох
DenseNet121 (ваша)	61.04%	102,500	только классификатор, 3 эпохи

--- Честные бейзлайны (обучение с нуля) ---

Модель	Точность	Параметры	Примечание
VGG-16	72.70%	20,000,000	с нуля
ResNet-18	75.60%	11,000,000	с нуля
ResNet-56	76.50%	24,000,000	с нуля
Wide ResNet-28-10	81.20%	36,000,000	с аугментацией

--- SOTA с предобучением (нельзя сравнивать напрямую) ---

Модель	Точность	Параметры	Примечание
DenseNet-BC (k=24)	82.80%	27,000,000	ImageNet pretrain
EfficientNet-B7	87.30%	66,000,000	ImageNet pretrain
FixResNeXt-101	89.90%	84,000,000	ImageNet pretrain
ViT-L/16	93.00%	307,000,000	JFT-300M pretrain
ViT-H/14	93.80%	632,000,000	JFT-300M pretrain

ИТОГИ

Лучшая ваша модель: 61.04%
Лучший честный бейзлайн: 81.20% (отставание: +20.16 п.п.)
Лучший SOTA (с предобучением): 93.80% (отставание: +32.76 п.п.)
Примечание: SOTA-модели используют предобучение на ImageNet/JFT-300M, поэтому их сравнение с моделями, обученными с нуля, не является строгим.

В ходе лабораторной работы обучена предобученная DenseNet121 на CIFAR-100 с оптимизатором Adam. Классификатор заменён на Linear(1024, 100) под число классов, изображения переформатированы с 32×32 до 224×224 с нормализацией ImageNet. Backbone заморожен, обучался только классификатор (102 500 параметров из 7 056 356). За 3 эпохи с CrossEntropyLoss достигнута точность 61.04% на тесте; графики показали устойчивое снижение ошибки и рост точности без переобучения. Кастомная SimpleCNN из ЛР №1 (643 492 параметра, 8 эпох с нуля) дала 37.23% - DenseNet121 превзошла её на 23.8 п.п. при в 6 раз меньшем числе обучаемых параметров, что подтверждает эффективность transfer learning. Сравнение с SOTA показало отставание на 14-20 п.п. от честных бейзлайнов (Wide ResNet-28-10 - 81.2%) и на 29-33 п.п. от моделей с предобучением (ViT-H/14 - 93.8%), что объясняется замороженным backbone, малым числом эпох и отсутствием сильных аугментаций. Реализована визуализация работы обеих сетей на произвольном изображении с отображением распределения вероятностей по классам.

Вывод: осуществила обучение НС, сконструированных на базе предобученных архитектур НС.